



<Bolsa Futuro Digital>

<aula 18>

Turma de FrontEnd – IF UFRGS

● Instalando algumas ferramentas



JavaScript foi criado para funcionar dentro do navegador, para rodar um código fora dele precisamos do **Node.js**

Também, para rodar os códigos dentro do VS Code vamos precisar de uma extensão, existem duas opções:

- **Node.js Exec**
- **Code Runner**



.run

● OneCompiler – JavaScript no navegador



OneCompiler

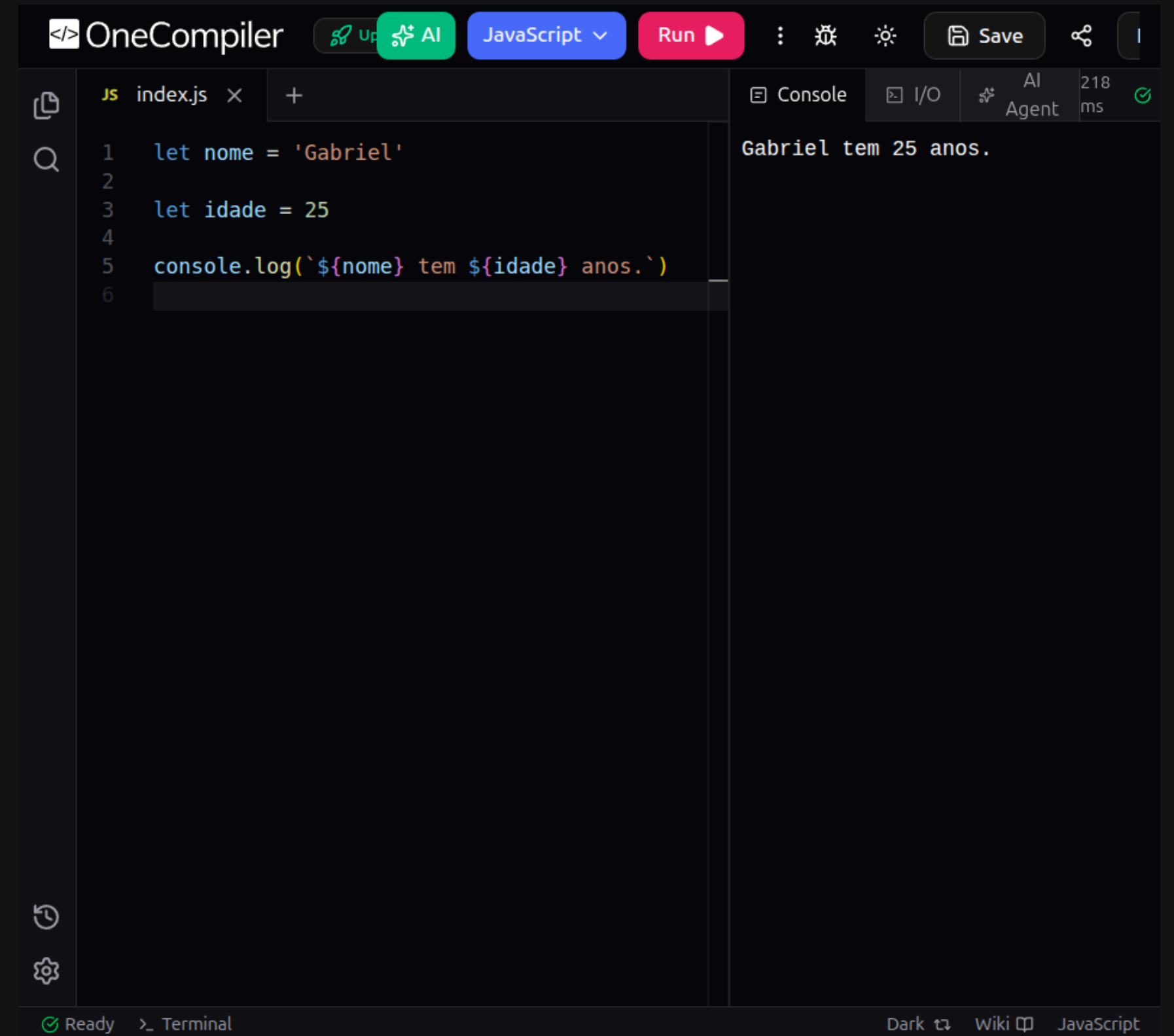
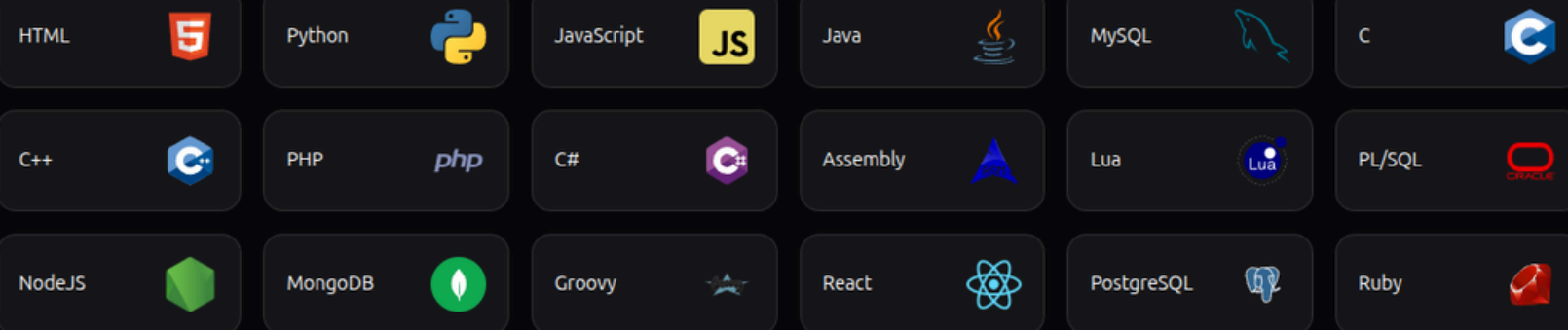
OneCompiler – JavaScript

Code online with **One Compiler.**

One Compiler helps over 12.8 million users worldwide write code online.

Q Search by Language/ DB/ Template etc.,

Popular Programming Web Databases



● JavaScript



HTML tem seu arquivo próprio no formato .html, o **CSS** podia ser aplicado dentro da tag <style> ou em um arquivo externo .css usando a tag <link>.

JavaScript funciona de forma parecida, podemos criar um script externo ou interno, mas ambos usam a tag <script>.

Geralmente usamos um código externo, assim como com o CSS, para manter tudo organizado.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    body {
      background-color: yellow;
      font-family: 'Times New Roman', Times, serif;
      font-size: 20pt;
    }
  </style>
</head>
<body>
  <h1>Meu Site</h1>
  <main>
    <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Quod
    minus ducimus laudantium voluptatibus. Deserunt magni pariatur
    facilis, vel fugit temporibus soluta officiis incidunt maxime
    distinctio quis nulla nostrum, a aspernatur.</p>
  </main>

  <!-- O Script vem sempre ao final do body -->
  <script>
    window.alert('Olá Mundo!')
  </script>
</body>
</html>
```

● Tipos de variáveis



Existem diferentes tipos de variáveis, dentre números, textos e valores booleanos (Verdadeiro ou Falso). Um texto é chamado de string e vem sempre entre 'aspas'. Os números podem ser int ou float e vem sem 'aspas'.

```
let texto = 'String' // Textos são chamados de strings
let numero = 10      // Number int - nº inteiro
let decimo = 10.20   // Number float - nº com vírgula (ponto!)
let boolean = true   // Boolean - true ou false

// typeof mostra o tipo da variável
console.log(typeof texto)
```


● Operadores matemáticos



```
let a = 10
let b = 2

// SOMA
console.log(a + b)
// SUBTRAÇÃO
console.log(a - b)
// MULTIPLICAÇÃO
console.log(a * b)
// DIVISÃO
console.log(a / b)
```

```
let a = 9
let b = 2

// O resto da divisão 9/2 é 1
console.log(a % b)
// 9^2 (ao quadrado) é 81
console.log(a ** b)

b++ // Incremento: b = b + 1 (adiciona um)
console.log(b)
b-- // Decremento: b = b - 1 (subtrai um)
console.log(b)
```

● Operadores lógicos



```
let a = 1  
let b = 5
```

```
// a é menor que b?  
console.log(a < b)  
// a é maior que b?  
console.log(a > b)  
// a é igual a b?  
console.log(a == b)  
// a é diferente de b?  
console.log(a != b)
```

```
let a = 2  
let b = '2'
```

```
// a é menor ou igual a b?  
console.log(a <= b)  
// a é maior ou igual a b?  
console.log(a >= b)  
// a e b são iguais  
// em valor e tipo?  
console.log(a === b)  
// a e b são diferentes  
// em valor ou tipo?  
console.log(a !== b)
```

● Operadores lógicos



Também existem os operadores lógicos usados quando fazemos mais de uma pergunta e o operador de negação:

`&&` (E / AND): ambas perguntas devem ser `true`

`||` (OU / OR): apenas uma pergunta precisa ser `true`

`!` (NÃO / NOT): troca `false` e `true`

```
// Operador && ( E / AND )
// Para ser true -> AMBOS DEVEM SER TRUE
let entrada = (idade >= 18) && (ingresso == true)

// Operador || ( OU / OR )
// Para ser true -> UM OU DOIS DEVEM SER TRUE
let entrada = (ingresso == true) || (convite == true)

// Operador ! ( NEGAÇÃO )
// Inverte um valor booleano
let comprar = !ingresso
```


● Condições



Até agora nossos códigos funcionaram de forma linear, ou seja, rodavam todas as linhas, uma após a outra.

qual a idade?



a idade é maior que 18?



entrada permitida

```
// qual a idade?  
let idade = 20  
  
// a idade é maior que 18?  
let entrada = idade >= 18  
  
// entrada permitida  
console.log('Entrada permitida!')
```

● Condições



As condições vão rodar as linhas apenas se uma pergunta for `true` ou `false`, dependendo do que você pedir

qual a idade?



a idade é maior que 18?

se sim (`true`)



entrada permitida

se não (`false`)



entrada negada

● Condições



`if` (significa se) e `else` (significa senão) são usados para criar uma condição, o código só é executado se a condição for satisfeita

```
if (condição) {  
    se satisfeita  
    faz isso  
} else {  
    se não for satisfeita  
    faz isso  
}
```

```
// qual a idade?  
let idade = 16  
  
// a idade é maior que 18?  
if (idade >= 18) {  
    // se sim, entrada permitida!  
    console.log('Entrada permitida!')  
} else {  
    // senão, entrada negada!  
    console.log('Entrada negada!')  
}
```

● Condições



Podemos adicionar condições adicionais combinando `else if`

```
if (condição) {  
    se satisfeita faz isso  
} else if (2ª condição) {  
    se satisfeita faz isso  
} else {  
    se nenhuma condição  
    for satisfeita faz isso  
}
```

```
// qual a idade?  
let idade = 16  
  
// a idade é maior que 18?  
if (idade >= 18) {  
    // se sim o voto é obrigatório!  
    console.log('Voto obrigatório')  
} else if ( idade >= 16 && idade < 18) {  
    // se for menor que 18, mas maior de 16  
    console.log('Voto opcional')  
} else {  
    // senão, o voto não é permitido  
    console.log('Voto não permitido')  
}
```

● Condições



Também é possível usar apenas o `if` sem o `else`, pois esse serve como um 'plano B' que as vezes não é necessário, se a condição não for satisfeita o código segue normalmente.

```
let saldo = 10

// Adiciona uma condição
if (saldo <= 5) {
  window.alert('ATENÇÃO! Seu saldo está baixo.')
}

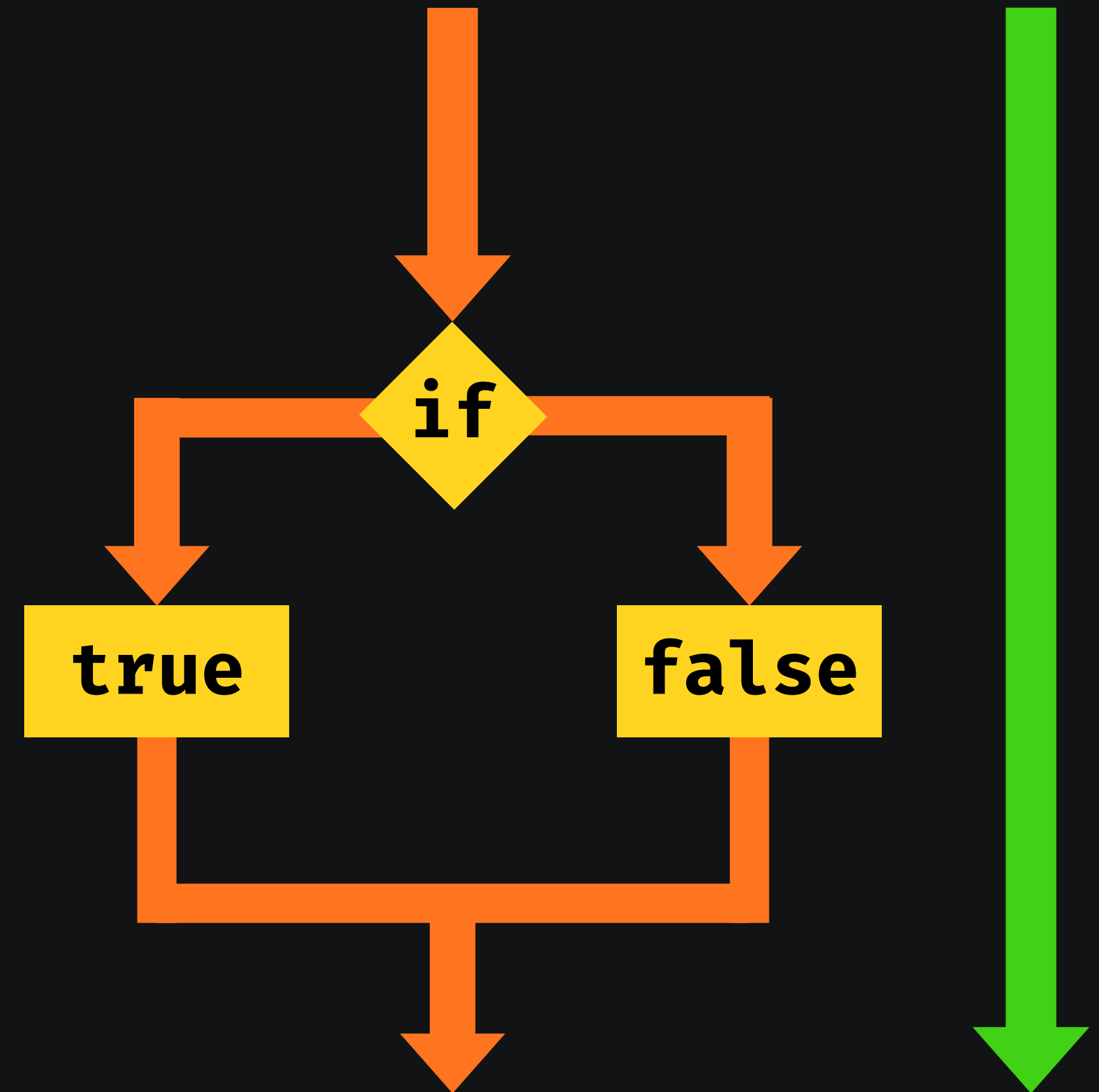
// Código continua normalmente...
```


● Repetições



Mesmo com as condições os nossos códigos ainda vão seguir uma linha, repetindo um comando de cada vez e uma ÚNICA vez.

Se quisermos repetir um comando precisamos repetir as linhas, mas isso é pouco prático, pois as vezes precisamos repetir um código várias vezes.



● Repetições



Enquanto a condição é satisfeita precisamos repetir a mesma linha até que a condição seja falsa.

```
let pos = 7
let fila = 1

if (fila == pos) {
  console.log(`Sua vez chegou!`)
} else {
  console.log(`Aguarde sua vez! Senha atual: ${fila}. Sua posição: ${pos}`)
}

fila ++ // fila = fila + 1

if (fila == pos) {
  console.log(`Sua vez chegou!`)
} else {
  console.log(`Aguarde sua vez! Senha atual: ${fila}. Sua posição: ${pos}`)
}

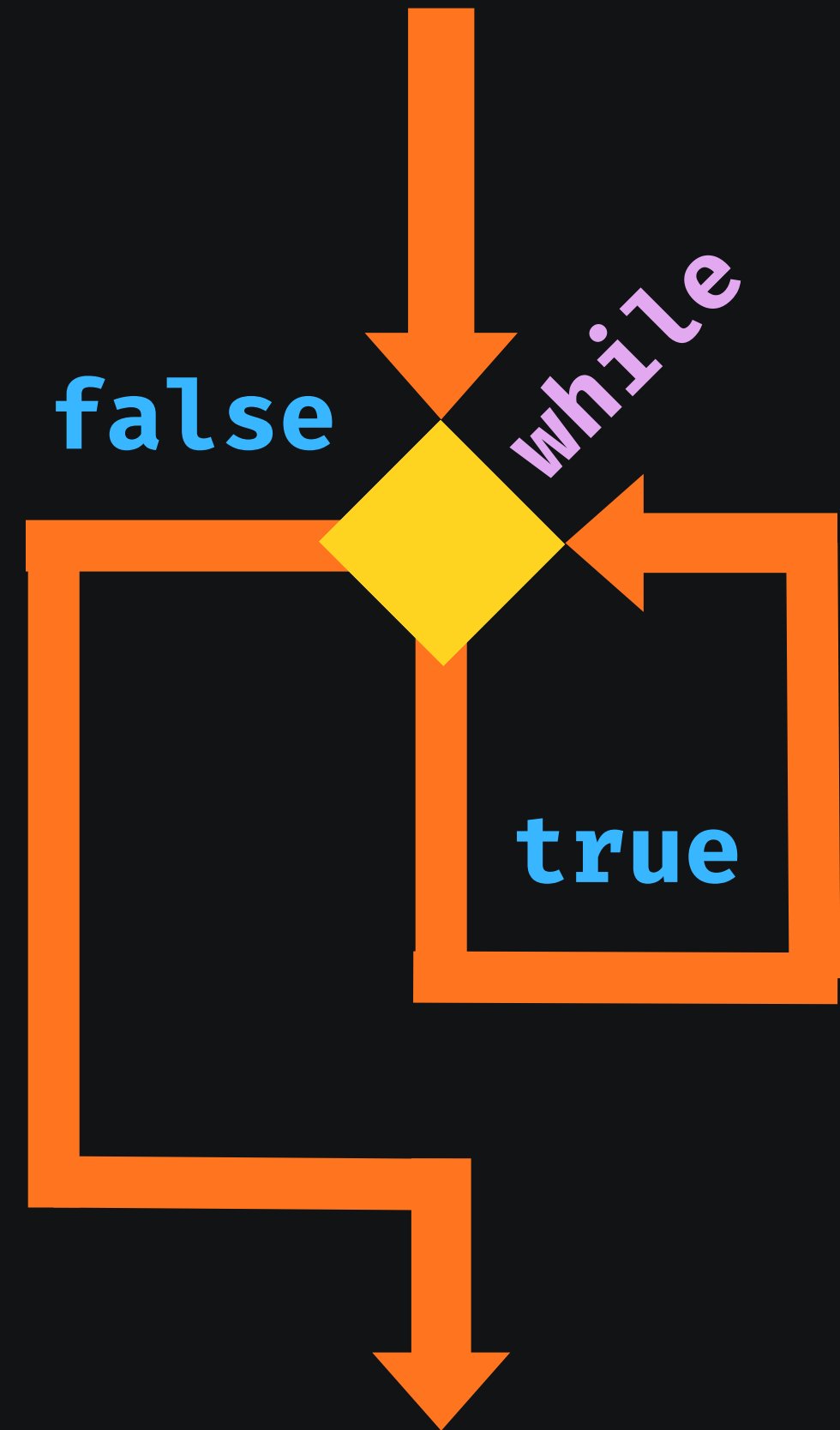
fila ++ // fila = fila + 1
```

● Repetições



```
Enquanto (a condição é true) {  
    repete essa linha  
}  
quando for false o código continua
```

```
while (true) {  
    repete essa linha  
}  
quando for false o código continua
```



● Repetições



```
Enquanto (a condição é true) {  
    repete essa linha  
}  
quando for false o código continua
```

```
let pos = 7  
let fila = 1  
  
while (fila < pos) {  
    console.log(`Aguarde sua vez! Senha atual: ${fila}. Sua posição: ${pos}`)  
    fila ++  
}  
  
console.log(`Sua vez chegou!`)
```

● Repetições



Outra forma de fazer uma repetição é usando um contador de passos, a cada repetição, ou seja, a cada passo do `while` você adiciona mais um ao contador.

contador começa em 0

```
Enquanto (o contador < n) {  
    repete essa linha  
    adiciona mais um ao contador  
}  
quando for maior o código continua
```

```
// Inicia um contador = 0  
let c = 0  
  
while(c <= 10) {  
    //contar até 10  
    console.log(c)  
    // c = c + 1  
    c++  
}  
  
console.log('Terminou!')
```

● Repetições



Uma forma muito mais prática de usar um contador de passos é com o `for`. Dentro do `for` definimos o passo inicial, final e o tamanho de cada passo.

```
Para (esse nº de passos) {  
    repete essa linha  
}  
quando terminar o código continua
```

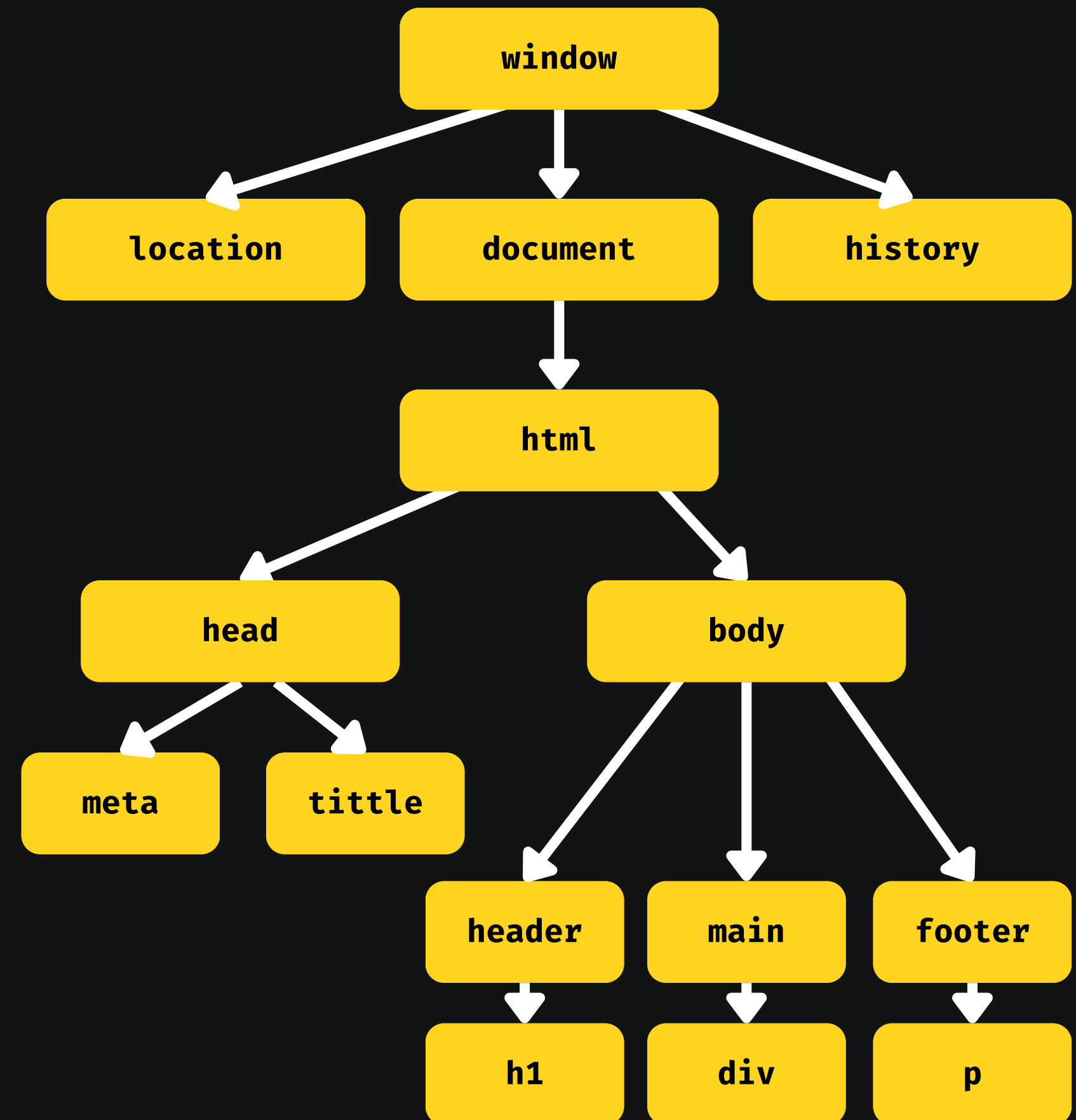
```
// (inicia em 1; vai até 10; de 1 em 1)  
for(let c=1; c<=10; c++) {  
    console.log(c)  
}  
console.log('Terminou!')
```

● DOM e a Árvore DOM



DOM (**D**ocument **O**bject **M**odel) é um modelo de objetos para documentos, basicamente é um conjunto de objetos dentro do seu navegador que vai dar acesso aos componentes internos da sua página.

O DOM só funciona dentro do navegador e vamos utilizá-lo no JavaScript para obter os elementos do arquivo HTML.



● Selecionando elementos com DOM



Então vamos usar o DOM para selecionar os elementos da nossa página. Podemos fazer a seleção com os seguintes métodos:

- **Marca:** `getElementsByTagName()`
- **ID:** `getElementById()`
- **Nome:** `getElementsByName()`
- **Classe:** `getElementsByClassName()`
- **Seletores:** `querySelector()`
`querySelectorAll()`

```
<script>

// Pega o primeiro parágrafo [0] e coloca em p1
let p1 = window.document.getElementsByTagName('p')[0]
// mostra na tela o valor de p1
window.document.write('Está escrito: ' + p1.innerText)
// .innerText traz apenas o texto
// .innerHTML traz o texto formatado

// Pega a div pelo id dela
let d = window.document.getElementById('msg')
// Modifica o estilo e conteúdo da div
d.style.background = 'green'
d.innerText = 'Aguardando...'

// Podemos usar a mesma identificação do CSS
let d = window.document.querySelector('div#msg')
d.style.background = 'green'
d.innerText = 'Aguardando...'

</script>
```




</aula 18>